OPERATING SYSTEMS · STUDY EDITION

শিক্ষকের হাতে লেখা নোট ব্যাখ্যা করে গড়া, নতুন শিক্ষার্থীর জন্য ধাপে ধাপে সমাধান

CPU Scheduling Algorithms

শূন্য থেকে সম্পূর্ণ বাংলা হ্যান্ডনোট | Operating Systems

পাঠের ধরন: ক্লাস নোটভিত্তিক ব্যাখ্যা + আলাদাভাবে চিহ্নিত অতিরিক্ত অনুশীলন
উপযোগিতা: বিশ্ববিদ্যালয়ের Operating Systems, সরকারি IT/Assistant Programmer ও ব্যাংক নিয়োগ পরীক্ষার প্রস্তুতি
মূল উৎস: সংযুক্ত শিক্ষকের হাতে লেখা `class-1(1).pdf`, পৃষ্ঠা ১–৫; পৃষ্ঠা ৬–৭-এ পাঠযোগ্য ক্লাস-নোটের বিষয়বস্তু নেই।
গুরুত্বপূর্ণ: এখানে কোথাও কোনো MCQ-কে নির্দিষ্ট নিয়োগ পরীক্ষার আগের বছরের প্রশ্ন হিসেবে দাবি করা হয়নি।

> **পড়ার কৌশল:** প্রথমে AT, BT, CT বুঝবে। তারপর `TAT = CT − AT`, `WT = TAT − BT` শিখবে। এরপর Gantt Chart তৈরি করে হিসাব করবে। সব অঙ্কে সময়ের একক একই ধরে নেওয়া হয়েছে, সাধারণভাবে `ms`।

১. শিক্ষকের নোটে আসলে কী আছে?

নোটের পৃষ্ঠা	শিক্ষক যা লিখেছেন/দেখিয়েছেন	এই হ্যান্ডনোটে ব্যবহার
১	Program, Process, HDD–RAM ধারণা; New, Ready, Running, Waiting, Terminated; Preemptive ও Non-Preemptive	ভিত্তি ও Process State চিত্র
২	Ready Queue, Arrival Time, Burst Time, Completion/Exit Time, Turnaround Time-এর উদাহরণ	পরিভাষা ও সূত্রের ব্যাখ্যা
৩	পাঁচটি Algorithm-এর নাম; **FCFS: Non-Preemptive**; P1=15, P2=4, P3=2-এর Gantt ও অপেক্ষার হিসাব	FCFS অঙ্ক ১
৪	Arrival Time-সহ FCFS-এর দুটি উদাহরণ; প্রথমটিতে CPU Idle এবং একসঙ্গে Arrival	FCFS অঙ্ক ২ ও ৩
৫	**SJF: Non-Preemptive**; Burst Time-ভিত্তিক ও Arrival Time-সহ দুটি Gantt	SJF অঙ্ক ১ ও ২
৬–৭	কোনো অতিরিক্ত পাঠযোগ্য অঙ্ক বা ব্যাখ্যা পাওয়া যায়নি	নতুন তথ্য ধরে নেওয়া হয়নি

শিক্ষকের নোটে নাম উল্লেখ করা পাঁচটি Algorithm: `FCFS`, `SJF`, `SRTF`, `Round Robin`, `Priority Scheduling`। কিন্তু **সম্পূর্ণ গাণিতিক উদাহরণ দেওয়া আছে কেবল FCFS ও Non-Preemptive SJF-এর**। তাই `SRTF`, `Priority` ও `Round Robin`-এর সমাধান **অতিরিক্ত শেখার উদাহরণ**, শিক্ষকের অঙ্ক নয়।

গুরুত্বপূর্ণ সতর্কতা: নোটের প্রথম FCFS ও প্রথম SJF উদাহরণে আলাদা Arrival Time দেওয়া হয়নি। প্রদর্শিত Gantt Chart অনুসারে **সবগুলোর AT = 0 ধরে** হিসাব সম্পূর্ণ করা হয়েছে। এটি নোটে অনুপস্থিত তথ্য ব্যাখ্যা করার জন্য করা স্পষ্ট অনুমান, শিক্ষক আলাদা করে AT লিখেছেন এমন দাবি নয়। প্রথম FCFS-এ P1, P2, P3-এর ক্রম নোটে দেখানো ক্রম।

সামান্য সংশোধন ও অস্পষ্টতা: (ক) `RAM-এ থাকা Program = Process` কথাটি প্রাথমিক ধারণা; বাস্তবে Process-এর CPU Register, Program Counter, Memory Context-ও থাকে। (খ) পৃষ্ঠা ২-এর সময়-সংক্রান্ত কথাগুলো ব্যাখ্যামূলক; সেগুলো সম্পূর্ণ Scheduling Dataset নয়। (গ) পৃষ্ঠা ৪-এ P1 ও P4 একই সময়ে আসে; নোটে P1 আগে চলে। এখানে সেই

নিয়ম ধরে **টেবিলে আগে থাকা Process আগে** নেওয়া হয়েছে। (ঘ) পৃষ্ঠা ৫-এর শেষ SJF অঙ্কে P1 ও P3-এর BT সমান; শিক্ষক আগে আসা P3 নিয়েছেন। এ নিয়মটিই অনুসরণ করা হয়েছে।

২. আজকের শেখার লক্ষ্য

এই পাঠ শেষে তুমি পারবে: (১) Process ও Ready Queue চেনা; (২) Non-Preemptive এবং Preemptive-এর পার্থক্য বলা; (৩) দেওয়া AT/BT থেকে Gantt Chart আঁকা; (৪) CT, TAT, WT, RT ও Average বের করা; (৫) FCFS, SJF, SRTF, Priority, Round Robin অঙ্কে সঠিক Process নির্বাচন করা; (৬) CPU Idle, Tie, Starvation ও Context Switch বুঝে ভুল এড়ানো।

৩. Program, Process ও CPU Scheduling

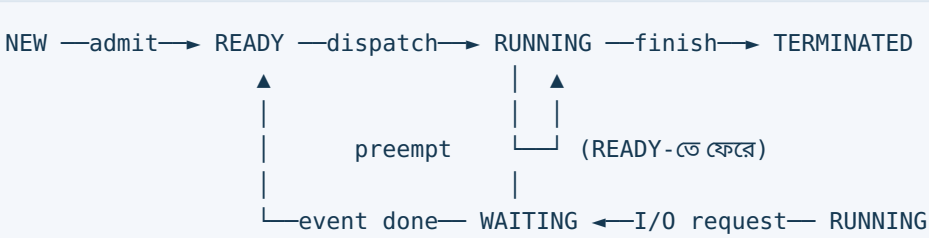
৩.১ Program বনাম Process

Program হলো সংরক্ষিত নির্দেশনার সমষ্টি। যেমন, কম্পিউটারে Calculator অ্যাপের ফাইল। **Process** হলো সেই Program চালু হলে তার কার্যরত অবস্থা। অর্থাৎ একই Program একাধিকবার চালু করলে একাধিক Process থাকতে পারে। **CPU** হলো প্রসেসরের সেই অংশ যা নির্দেশনা সম্পাদন করে।

সহজ উদাহরণ: একটি দোকানে তিনজন ক্রেতার বিল তৈরি করতে হবে। ক্যাশিয়ার একজন; ক্রেতা তিনজন। ক্যাশিয়ার যেমন ঠিক করেন কার বিল আগে করবেন, তেমনি Operating System-এর **CPU Scheduler** নির্ধারণ করে Ready Queue-এ থাকা কোন Process CPU পাবে।

CPU Scheduling: একাধিক প্রস্তুত Process-এর মধ্যে পরবর্তী CPU-ব্যবহারকারী বেছে নেওয়ার পদ্ধতি। **Ready Queue** হলো CPU পাওয়ার অপেক্ষায় থাকা প্রস্তুত Process-গুলোর সারি। **Dispatcher** হলো Scheduler-এর নির্বাচিত Process-কে CPU ব্যবহারের নিয়ন্ত্রণ দেওয়ার ব্যবস্থা।

৩.২ Process-এর পাঁচটি সাধারণ State



State	বাংলায় অর্থ	সহজ উদাহরণ
New	Process তৈরি হচ্ছে	নতুন অ্যাপ চালু করার শুরু
Ready	CPU পাওয়ার জন্য প্রস্তুত	সব প্রস্তুত, শুধু CPU-এর অপেক্ষা
Running	CPU-তে কাজ চলছে	এখন গণনা করছে
Waiting / Blocked	কোনো Event/I/O শেষ হওয়ার অপেক্ষা	ফাইল পড়ার অপেক্ষা
Terminated	কাজ শেষ	অ্যাপের কাজ শেষ/বন্ধ

মনে রেখো: `Ready` মানে Process CPU পায়নি; `Waiting` মানে CPU পেলেই কাজ শুরু করতে পারবে এমন নয়, আগে কোনো Event বা I/O শেষ হতে হবে।

৩.৩ Non-Preemptive বনাম Preemptive

বৈশিষ্ট্য	Non-Preemptive	Preemptive
মূল নিয়ম	CPU পাওয়া Process তার CPU Burst শেষ করা বা নিজে ব্লক করা পর্যন্ত চলে	চলমান Process-এর CPU কেড়ে অন্য Process চালানো যায়
সিদ্ধান্তের মুহূর্ত	CPU খালি হলে বা Running Process ব্লক করলে	নতুন Process এলে, Time Quantum শেষ হলে অথবা অন্য Scheduling Event-এ
সহজ উদাহরণ	ক্যাশিয়ার একটি বিল শেষ করে পরের বিল ধরেন	জরুরি ক্রেতাকে আগে সেবা দিতে চলতি কাজ থামানো হয়
উদাহরণ	FCFS, Non-Preemptive SJF	SRTF, Round Robin, Preemptive Priority

Preemption মানে হলো একটি Process শেষ হওয়ার আগেই CPU থেকে সরিয়ে দেওয়া। **Context Switch** মানে বর্তমান Process-এর প্রয়োজনীয় অবস্থা সংরক্ষণ করে অন্যটির অবস্থা ফিরিয়ে CPU বদলানো; এতে কিছু অতিরিক্ত সময় খরচ হয়। এই হ্যান্ডনোটের অঙ্কগুলোতে আলাদা Context Switch Overhead না দেওয়া থাকায় সেটি `0` ধরা হয়েছে।

৪. অঙ্ক শুরুর আগে প্রতিটি শব্দ বোঝো

সংক্ষিপ্ত রূপ	পূর্ণ নাম	সহজ বাংলা অর্থ
PID	Process ID	P1, P2-এর মতো Process-এর পরিচয়
AT	Arrival Time	Process কখন Ready Queue-তে এলো
BT	Burst Time	CPU-তে মোট কত সময় কাজ করতে হবে
ST	Start Time / First Start	প্রথমবার কখন CPU পেল
CT	Completion Time	পুরো CPU কাজ শেষ হওয়ার সময়
TAT	Turnaround Time	আসা থেকে সম্পূর্ণ শেষ হওয়া পর্যন্ত মোট সময়
WT	Waiting Time	Ready অবস্থায় CPU-এর জন্য মোট অপেক্ষা
RT	Response Time	প্রথম CPU পাওয়া পর্যন্ত অপেক্ষা
TQ বা q	Time Quantum	Round Robin-এ একবারে সর্বোচ্চ CPU সময়
Priority	অগ্রাধিকারের মান	কোন Process আগে CPU পাবে তার একটি নিয়মভিত্তিক মান

৪.১ ছোট উদাহরণ: সময়ের মান কোথা থেকে আসে?

ধরো P0 সময় `0–5` পর্যন্ত CPU ব্যবহার করছে। এর মধ্যে P1 `AT=2`-তে এসেছে। P0 শেষ হলে P1 `ST=5`-এ CPU পেয়েছে, `BT=4` সময় কাজ করে `CT=9`-এ শেষ হয়েছে।

Gantt Chart 1: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

- **AT = 2:** Process এসেছে সময় ২-এ।

- **BT = 4:** CPU-তে কাজ করেছে `9 − 5 = 4` সময়।
- **CT = 9:** কাজ শেষ সময় ৯-এ।
- **TAT = 9 − 2 = 7:** আসার পর মোট ৭ সময়ের মধ্যে কাজ শেষ।
- **WT = 7 − 4 = 3:** মোট ৭ সময়ের মধ্যে CPU-তে কাজ ৪; তাই অপেক্ষা ৩।
- **RT = 5 − 2 = 3:** প্রথম CPU পেতে ৩ সময় লেগেছে।

খেয়াল করো: P1 যখন `t=2`-এ এসেছে, তখন P0 CPU-তে চলছে। তাই P1 `2–5` সময় অপেক্ষা করে। এর আগে P1 আসেইনি বলে `0–2` তার Waiting Time নয়।

৪.২ পরীক্ষার মূল Formula Box

```
TAT = CT − AT
WT  = TAT − BT
RT  = First Start Time − AT
Average TAT = (সব Process-এর TAT-এর যোগফল) / Process সংখ্যা
Average WT  = (সব Process-এর WT-এর যোগফল) / Process সংখ্যা
Average RT  = (সব Process-এর RT-এর যোগফল) / Process সংখ্যা
```

কেন WT = TAT − BT? Process মোট যত সময় System-এ ছিল, তার মধ্যে CPU-তে কাজের সময় বাদ দিলে CPU পাওয়ার অপেক্ষার সময় থাকে। এখানে কেবল **একটি CPU Burst, কোনো আলাদা I/O Waiting নেই** এমন প্রচলিত পরীক্ষার মডেল ধরা হয়েছে। একাধিক CPU/I/O Burst থাকলে Waiting Time হিসাবের জন্য Ready Queue-এ কাটানো সময় আলাদা করে যোগ করতে হয়; কেবল `TAT − মোট CPU BT` করলে I/O-এর অপেক্ষাও যুক্ত হয়ে যেতে পারে।

RT-এর বিশেষ নিয়ম: Non-Preemptive একক CPU Burst সমস্যায় সাধারণত `RT = WT`। কিন্তু Preemptive সমস্যায় Process মাঝপথে আবার অপেক্ষা করতে পারে; তাই `RT` এবং `WT` এক নাও হতে পারে।

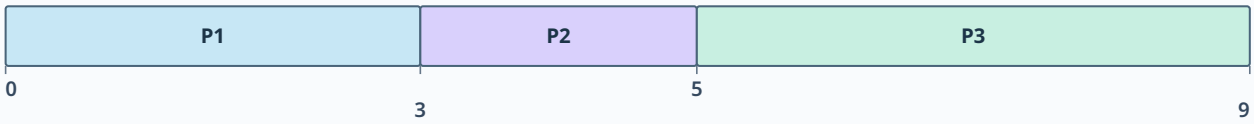
৪.৩ CPU Idle, Throughput ও Utilization

- **CPU Idle:** Ready Queue-তে চালানোর মতো Process নেই বলে CPU ফাঁকা থাকে। Gantt-এ `Idle` লিখবে।
- **Throughput:** নির্দিষ্ট সময়ে কতটি Process শেষ হলো। সূত্র: `Completed Processes / Elapsed Time`।
- **CPU Utilization:** `CPU Busy Time / Total Observed Time × 100%`। যেমন, ০–১০ সময়ের মধ্যে ৮ সময় CPU কাজ করলে Utilization `80%`।
- **Scheduling-এর লক্ষ্য:** Waiting/Turnaround/Response Time সাধারণত কমানো এবং CPU Utilization/Throughput সাধারণত বাড়ানো; কিন্তু সব লক্ষ্য একই সঙ্গে সর্বোত্তম হওয়া সব সময় সম্ভব নয়।

৫. Gantt Chart আঁকার সঠিক পদ্ধতি

Gantt Chart হলো সময়ের রেখায় কোন Process কখন CPU পেয়েছে তার ছবি। প্রতিটি Block-এর নিচে শুরুর ও শেষের সময় থাকে। যেমন:

Gantt Chart 2: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

পড়ার নিয়ম: P1 চলেছে `0–3` অর্থাৎ ৩ সময়; P2 চলেছে `3–5` অর্থাৎ ২ সময়; P3 চলেছে `5–9` অর্থাৎ ৪ সময়। প্রতিটি Block-এর দৈর্ঘ্য হলো `শেষ সময় − শুরুর সময়`। প্রথম উদাহরণে P2-এর BT হলো ২, ৫ নয়। **Gantt-এ ৫ তার Completion Time, Burst Time নয়।**

প্রতিটি অঙ্কের ছয় ধাপ: (১) AT, BT, Priority/TQ দেখো; (২) সময় `t`-এ কারা এসেছে যাচাই করো; (৩) Algorithm-এর নিয়মে একজন নির্বাচন করো; (৪) Gantt-এ সঠিক সময় বসাও; (৫) Process-এর শেষ Block থেকে CT লেখো; (৬) TAT, WT, RT ও Average বের করো।

অংশ A — শিক্ষকের নোটভিত্তিক সমাধান

৬. FCFS — First Come, First Served

FCFS অর্থ আগে আসলে আগে CPU পাবে। এটি সাধারণত **Non-Preemptive**। Ready Queue-এ যে আগে ঢুকেছে, সে আগে চালু হবে। **FIFO (First In, First Out)** Queue-এর নিয়ম অনুসরণ করে।

বাস্তব উদাহরণ: দোকানে আগে লাইনে দাঁড়ানো ব্যক্তি আগে সেবা পান, তার কাজ বড় হলেও মাঝখানে কাজ থামানো হয় না।

সুবিধা: সহজ, বাস্তবায়ন কম জটিল, Ready Queue-এ আগে আসা Process-কে অন্যরা অকারণে পেছনে ঠেলে দেয় না।
অসুবিধা: বড় Burst Time-এর Process সামনে থাকলে ছোটগুলো দীর্ঘক্ষণ অপেক্ষা করে; একে **Convoy Effect** বলে। FCFS সাধারণত Interactive কাজের Response Time ভালো দেয় না।

৬.১ FCFS অঙ্ক ১ — শিক্ষকের নোট, পৃষ্ঠা ৩

দেওয়া আছে: নোটে BT দেওয়া এবং ক্রম P1 → P2 → P3। AT লেখা নেই। নোটের Gantt অনুসারে সবার `AT=0` ধরে নিচ্ছি।

Process	AT (ধরা হয়েছে)	BT
P1	0	15
P2	0	4
P3	0	2

কেন P1 আগে? সবাই সময় ০-এ উপস্থিত ধরে নেওয়া হয়েছে; সমান Arrival-এর ক্ষেত্রে নোটে দেখানো তালিকার ক্রম P1, P2, P3 অনুসরণ করছি। FCFS ছোট BT দেখে Process বদলায় না।

সিদ্ধান্ত ১: সময় ০-এ P1 শুরু। ১৫ সময় চলবে। `0 + 15 = 15`।
সিদ্ধান্ত ২: সময় ১৫-তে P2 শুরু। `15 + 4 = 19`।
সিদ্ধান্ত ৩: সময় ১৯-এ P3 শুরু। `19 + 2 = 21`।

Gantt Chart 3: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT = CT−AT	WT = TAT−BT	RT = ST−AT
P1	0	15	0	15	15−0 = **15**	15−15 = **0**	0−0 = **0**
P2	0	4	15	19	19−0 = **19**	19−4 = **15**	15−0 = **15**
P3	0	2	19	21	21−0 = **21**	21−2 = **19**	19−0 = **19**

Average TAT: `(15 + 19 + 21) / 3 = 55 / 3 = 18.33`।
Average WT: `(0 + 15 + 19) / 3 = 34 / 3 = 11.33`।
Average RT: `(0 + 15 + 19) / 3 = 11.33`।

ব্যাখ্যা: P3-এর BT মাত্র ২; তবু তাকে ১৯ সময় অপেক্ষা করতে হয়েছে। কারণ P1 ও P2 আগে এসেছে বলে ধরে নেওয়া ক্রম অনুযায়ী CPU পেয়েছে। এটাই Convoy Effect বোঝার ভালো উদাহরণ।

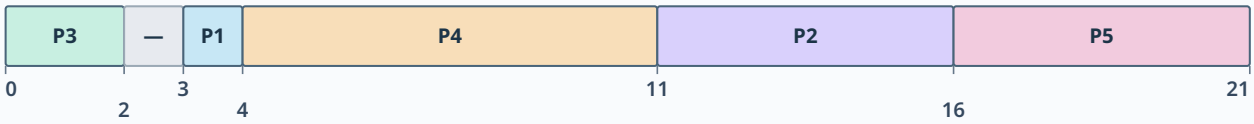
৬.২ FCFS অঙ্ক ২ — শিক্ষকের নোট, পৃষ্ঠা ৪ (Idle ও Tie)

Process	AT	BT
P1	3	1
P2	4	5
P3	0	2
P4	3	7
P5	5	5

Tie-Breaking নিয়ম: P1 ও P4 দুজনের AT=3। নোটের চিত্রে P1 আগে আছে; তাই এই অঙ্কে তালিকায় আগে থাকা P1 আগে যাবে। অন্য Tie Rule দিলে ফল বদলাতে পারে।

সিদ্ধান্ত ১: `t=0`-এ শুধু P3 এসেছে (`AT=0`)। P3 চলে `0–2`।
সিদ্ধান্ত ২: `t=2`-এ কোনো নতুন Process আসেনি। পরবর্তী Arrival `t=3`। তাই CPU **Idle 2–3**।
সিদ্ধান্ত ৩: `t=3`-এ P1 ও P4 দুজন আসে। Tie Rule অনুযায়ী P1, চলে `3–4`।
সিদ্ধান্ত ৪: `t=4`-এ P2 এসেছে, কিন্তু P4-এর AT=3; তাই আগে অপেক্ষারত P4 চলে `4–11`।
সিদ্ধান্ত ৫: `t=11`-এ P2 (AT=4), P5 (AT=5) অপেক্ষায়। P2 আগে এসেছে, চলে `11–16`।
সিদ্ধান্ত ৬: শেষ P5 চলে `16–21`।

Gantt Chart 4: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	3	1	3	4	4−3 = **1**	1−1 = **0**	3−3 = **0**
P2	4	5	11	16	16−4 = **12**	12−5 = **7**	11−4 = **7**
P3	0	2	0	2	2−0 = **2**	2−2 = **0**	0−0 = **0**
P4	3	7	4	11	11−3 = **8**	8−7 = **1**	4−3 = **1**
P5	5	5	16	21	21−5 = **16**	16−5 = **11**	16−5 = **11**

Average TAT: `(1+12+2+8+16)/5 = 39/5 = 7.80`।
Average WT: `(0+7+0+1+11)/5 = 19/5 = 3.80`।
Average RT: `(0+7+0+1+11)/5 = 3.80`।

CPU Idle: `3 − 2 = 1` সময়। মোট `0–21` পর্যবেক্ষণে CPU Busy `1+5+2+7+5 = 20` সময়; তাই **CPU Utilization = 20/21 × 100 ≈ 95.24%** (Context Switch বাদে)।

কঠিন জায়গাটি বোঝো: `t=4`-এ P2 এসেছে বলে তাকে চালানো যাবে না। P4 আগে `t=3`-এ এসেছে। FCFS-এ Arrival Order আগে, Burst Time নয়।

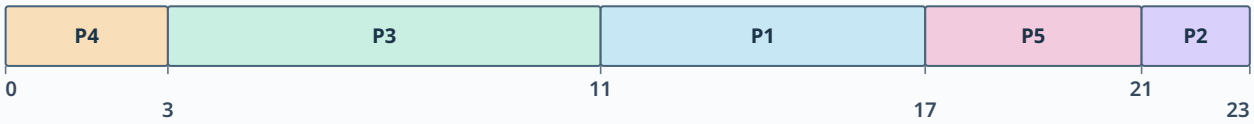
৬.৩ FCFS অঙ্ক ৩ — শিক্ষকের নোট, পৃষ্ঠা ৪

Process	AT	BT
P1	2	6
P2	5	2
P3	1	8
P4	0	3
P5	4	4

প্রথমেই AT সাজাও: P4(0) → P3(1) → P1(2) → P5(4) → P2(5)। **এই ক্রমেই FCFS চলবে।**

সিদ্ধান্ত ১: P4: `0–3` (`0+3=3`)।
সিদ্ধান্ত ২: P3: `3–11` (`3+8=11`)। P1 এর মধ্যে এলেও P3 থামবে না।
সিদ্ধান্ত ৩: P1: `11–17` (`11+6=17`)।
সিদ্ধান্ত ৪: P5: `17–21` (`17+4=21`)।
সিদ্ধান্ত ৫: P2: `21–23` (`21+2=23`)।

Gantt Chart 5: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	2	6	11	17	17−2 = **15**	15−6 = **9**	11−2 = **9**
P2	5	2	21	23	23−5 = **18**	18−2 = **16**	21−5 = **16**
P3	1	8	3	11	11−1 = **10**	10−8 = **2**	3−1 = **2**
P4	0	3	0	3	3−0 = **3**	3−3 = **0**	0−0 = **0**
P5	4	4	17	21	21−4 = **17**	17−4 = **13**	17−4 = **13**

Average TAT: `(15+18+10+3+17)/5 = 63/5 = 12.60`।
Average WT: `(9+16+2+0+13)/5 = 40/5 = 8.00`।
Average RT: `40/5 = 8.00`।

দেখো: P2-এর BT মাত্র ২ হলেও AT=5, কিন্তু আগে আসা Process-গুলো শেষ না হওয়া পর্যন্ত অপেক্ষা করেছে।

৭. SJF — Shortest Job First (Non-Preemptive)

SJF অর্থ Ready Queue-এ থাকা Process-গুলোর মধ্যে যার **Burst Time সবচেয়ে ছোট**, তাকে CPU দেওয়া। শিক্ষকের নোটে এই সংস্করণটি **Non-Preemptive**: একবার শুরু করলে BT শেষ না হওয়া পর্যন্ত থামবে না।

অত্যন্ত গুরুত্বপূর্ণ: `t=0`-এ যে Process এখনো আসেনি, তাকে ছোট BT হলেও নেওয়া যাবে না। শুধু **বর্তমানে এসে Ready থাকা** Process-গুলোর মধ্যে তুলনা করবে।

সুবিধা: সব Process একই সময়ে পাওয়া গেলে মোট/গড় অপেক্ষার সময় কমাতে কার্যকর; অনেক Workload-এ FCFS-এর চেয়ে ভালো Average WT দেয়। **সীমাবদ্ধতা:** আগেই Burst Time জানা কঠিন হতে পারে; দীর্ঘ কাজ নতুন ছোট কাজের কারণে বারবার পিছিয়ে যেতে পারে (**Starvation**, অর্থ দীর্ঘক্ষণ CPU না পাওয়া)।

৭.১ SJF অঙ্ক ১ — শিক্ষকের নোট, পৃষ্ঠা ৫

দেওয়া আছে: নোটে AT নেই। Gantt অনুযায়ী সবার `AT=0` ধরে নিচ্ছি।

Process	AT (ধরা হয়েছে)	BT
P1	0	15
P2	0	4
P3	0	2

সিদ্ধান্ত ১: `t=0`-এ সবাই Ready। সবচেয়ে ছোট BT হলো P3-এর ২। P3 চলে `0–2`।
সিদ্ধান্ত ২: বাকি P1=15, P2=4। ছোট P2, চলে `2–6`।
সিদ্ধান্ত ৩: বাকি P1=15, চলে `6–21`।

Gantt Chart 6: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	0	15	6	21	21−0 = **21**	21−15 = **6**	6−0 = **6**
P2	0	4	2	6	6−0 = **6**	6−4 = **2**	2−0 = **2**
P3	0	2	0	2	2−0 = **2**	2−2 = **0**	0−0 = **0**

Average TAT: `(21+6+2)/3 = 29/3 = 9.67`।
Average WT: `(6+2+0)/3 = 8/3 = 2.67`।
Average RT: `8/3 = 2.67`।

একই Dataset-এ তুলনা: FCFS-এ Average WT `11.33`, SJF-এ `2.67`। তাই এই নির্দিষ্ট উদাহরণে SJF কম অপেক্ষা করিয়েছে। এটি সব Scheduling লক্ষ্যেই সব সময় সেরা, এমন নয়।

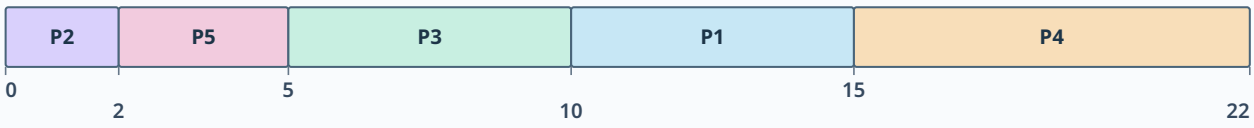
৭.২ SJF অঙ্ক ২ — শিক্ষকের নোট, পৃষ্ঠা ৫ (AT ও BT Tie)

Process	AT	BT
P1	4	5
P2	0	2
P3	1	5
P4	6	7
P5	2	3

এই অঙ্কে Tie Rule: BT সমান হলে **যার AT আগে**, তাকে আগে নেওয়া হবে। AT-ও সমান হলে তালিকার ক্রম।

সিদ্ধান্ত ১, t=0: শুধু P2 এসেছে। তাই `P2: 0–2`।
সিদ্ধান্ত ২, t=2: P3 (BT=5) এবং P5 (BT=3) Ready। P5 ছোট, তাই `P5: 2–5`।
সিদ্ধান্ত ৩, t=5: P3 (AT=1, BT=5) এবং P1 (AT=4, BT=5) Ready। BT সমান। আগে এসেছে P3; তাই `P3: 5–10`।
সিদ্ধান্ত ৪, t=10: P1 (BT=5) ও P4 (BT=7) Ready। P1 ছোট, তাই `P1: 10–15`।
সিদ্ধান্ত ৫: অবশিষ্ট P4, চলে `15–22`।

Gantt Chart 7: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	4	5	10	15	15−4 = **11**	11−5 = **6**	10−4 = **6**
P2	0	2	0	2	2−0 = **2**	2−2 = **0**	0−0 = **0**
P3	1	5	5	10	10−1 = **9**	9−5 = **4**	5−1 = **4**
P4	6	7	15	22	22−6 = **16**	16−7 = **9**	15−6 = **9**
P5	2	3	2	5	5−2 = **3**	3−3 = **0**	2−2 = **0**

Average TAT: `(11+2+9+16+3)/5 = 41/5 = 8.20`।
Average WT: `(6+0+4+9+0)/5 = 19/5 = 3.80`।
Average RT: `19/5 = 3.80`।

সবচেয়ে সাধারণ ভুল: `t=0`-এ পুরো টেবিল দেখে P5-এর BT=3-কে P2-এর পরে রাখা ঠিক, কিন্তু P5 প্রথমে চলতে পারে না; কারণ P5 আসে `t=2`-এ। আবার `t=5`-এ BT সমান হওয়ায় P3-এর আগে P1 নেওয়া এই নোটে ব্যবহৃত Tie Rule-এর সঙ্গে মেলে না।

অংশ B — নোটে শুধু নাম আছে; বিস্তারিত অতিরিক্ত শেখার অনুশীলন

৮. SRTF — Shortest Remaining Time First (Preemptive SJF)

SRTF হলো SJF-এর **Preemptive** সংস্করণ। CPU-তে থাকা Process-এর **Remaining Time** (আর কত সময় CPU লাগবে) এবং নতুন আসা Process-এর Burst Time তুলনা করা হয়। নতুন Process-এর প্রয়োজনীয় সময় **কঠোরভাবে কম** হলে চলমান কাজ থামিয়ে তাকে নেওয়া যায়।

Remaining Time = Original BT − ইতিমধ্যে CPU-তে চলার সময়। এই অঙ্কে Remaining Time সমান হলে বর্তমান Process চালিয়ে যাওয়ার Tie Rule ব্যবহার করব।

সুবিধা: ছোট কাজ দ্রুত শেষ হতে পারে। **অসুবিধা:** বারবার Context Switch, বড় Process-এর Starvation-এর ঝুঁকি, Remaining Burst আন্দাজের প্রয়োজন।

৮.১ SRTF সম্পূর্ণ সমাধান — অতিরিক্ত তৈরি উদাহরণ

Process	AT	BT
P1	0	7
P2	2	4
P3	4	1
P4	5	4

সিদ্ধান্ত ১, t=0: শুধু P1, চলে `0–2`। P1-এর অবশিষ্ট `7−2 = 5`।
সিদ্ধান্ত ২, t=2: P2 এসেছে, BT=4। P1-এর Remaining=5। `4 < 5`, তাই **P1 Preempt**, P2 শুরু।
সিদ্ধান্ত ৩, t=4: P2 চলেছে `2–4`, Remaining= `4−2=2`। P3 এসেছে BT=1। `1 < 2`, তাই **P2 Preempt**, P3 চলে `4–5`,

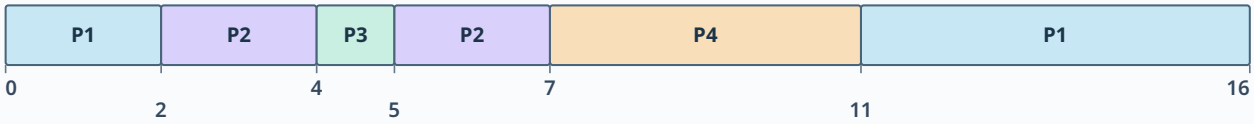
শেষ।

সিদ্ধান্ত ৪, t=5: P4 এসেছে BT=4। অপেক্ষায় P1 Remaining=5, P2 Remaining=2, P4=4। সবচেয়ে কম P2=2; P2 চলে `5–7`, শেষ।

সিদ্ধান্ত ৫, t=7: অপেক্ষায় P1 Remaining=5, P4=4। P4 ছোট; চলে `7–11`, শেষ।

সিদ্ধান্ত ৬, t=11: P1-এর বাকি ৫; চলে `11–16`, শেষ।

Gantt Chart 8: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	0	7	0	16	16−0 = **16**	16−7 = **9**	0−0 = **0**
P2	2	4	2	7	7−2 = **5**	5−4 = **1**	2−2 = **0**
P3	4	1	4	5	5−4 = **1**	1−1 = **0**	4−4 = **0**
P4	5	4	7	11	11−5 = **6**	6−4 = **2**	7−5 = **2**

Average TAT: `(16+5+1+6)/4 = 28/4 = 7.00`।

Average WT: `(9+1+0+2)/4 = 12/4 = 3.00`।

Average RT: `(0+0+0+2)/4 = 2/4 = 0.50`।

RT আর WT কেন আলাদা? P1 প্রথমে CPU পেয়েছে, তাই RT=0; কিন্তু পরে একাধিকবার অন্য Process চলায় তার মোট WT=9।

৯. Priority Scheduling — অগ্রাধিকারভিত্তিক CPU নির্বাচন

Priority হলো প্রত্যেক Process-কে দেওয়া গুরুত্বসূচক মান। **সতর্কতা:** কোনো বই/প্রশ্নে ছোট Number বেশি Priority, আবার কোথাও বড় Number বেশি Priority। প্রশ্নের নিয়ম না দেখে সিদ্ধান্ত নিও না। এই হ্যান্ডনোটের দুটি উদাহরণে **ছোট Number = বেশি Priority**।

Priority Scheduling **Non-Preemptive বা Preemptive—দুই ধরনেরই** হতে পারে। Priority সমান হলে আমরা আগে Arrival এবং তারপর Process তালিকার ক্রম অনুসরণ করব। দীর্ঘদিন Low Priority Process অপেক্ষায় থাকলে **Aging** (অপেক্ষা বাড়লে Priority বাড়ানো) পদ্ধতিতে Starvation কমানো যায়।

৯.১ Non-Preemptive Priority — অতিরিক্ত তৈরি উদাহরণ

Process	AT	BT	Priority
P1	0	4	2
P2	1	3	1
P3	2	2	3

Process	AT	BT	Priority
P4	3	1	1

সিদ্ধান্ত ১, t=0: শুধু P1 Ready। Priority মাঝারি হলেও অন্য কেউ নেই; P1 চলে `0–4`।

সিদ্ধান্ত ২, t=4: P2(pr=1), P3(pr=3), P4(pr=1) এসেছে। P2 ও P4-এর Priority সমান সবচেয়ে বেশি। P2 আগে এসেছে (`AT=1 < 3`), তাই `P2: 4–7`।

সিদ্ধান্ত ৩: বাকি P4(pr=1) ও P3(pr=3); আগে P4, `7–8`।

সিদ্ধান্ত ৪: শেষে P3, `8–10`।

Gantt Chart 9: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	Priority	First Start	CT	TAT	WT	RT
P1	0	4	2	0	4	4−0 = **4**	4−4 = **0**	0−0 = **0**
P2	1	3	1	4	7	7−1 = **6**	6−3 = **3**	4−1 = **3**
P3	2	2	3	8	10	10−2 = **8**	8−2 = **6**	8−2 = **6**
P4	3	1	1	7	8	8−3 = **5**	5−1 = **4**	7−3 = **4**

Average TAT: `(4+6+8+5)/4 = 23/4 = 5.75`।

Average WT: `(0+3+6+4)/4 = 13/4 = 3.25`।

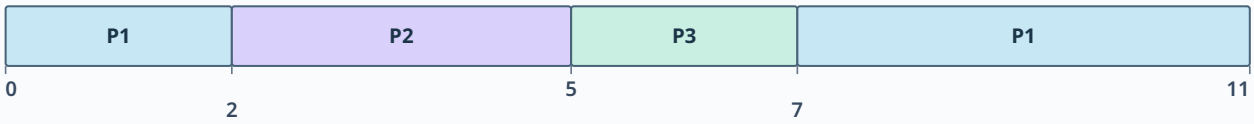
Average RT: `13/4 = 3.25`।

৯.২ Preemptive Priority — অতিরিক্ত তৈরি ছোট উদাহরণ

Process	AT	BT	Priority
P1	0	6	3
P2	2	3	1
P3	3	2	2

সিদ্ধান্ত: `0–2` P1 চলল, Remaining=4। `t=2`-এ P2 এসেছে Priority=1, যা P1-এর Priority=3-এর চেয়ে বেশি; তাই P1 থামল। P2 `2–5` চলল। `t=3`-এ P3 এলেও P2-এর Priority আরও বেশি, তাই থামবে না। এরপর `5–7` P3; সবশেষে `7–11` P1-এর বাকি ৪।

Gantt Chart 10: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	0	6	0	11	11−0 = **11**	11−6 = **5**	0−0 = **0**
P2	2	3	2	5	5−2 = **3**	3−3 = **0**	2−2 = **0**
P3	3	2	5	7	7−3 = **4**	4−2 = **2**	5−3 = **2**

Average TAT: `(11+3+4)/3 = 18/3 = 6.00`।
Average WT: `(5+0+2)/3 = 7/3 = 2.33`।
Average RT: `(0+0+2)/3 = 2/3 = 0.67`।

১০. Round Robin (RR) — পালাক্রমে নির্দিষ্ট সময়

Round Robin-এ প্রতিটি Ready Process সর্বোচ্চ **Time Quantum (`TQ` বা `q`)** সময় CPU পায়। Process আগে শেষ হলে সঙ্গে সঙ্গে CPU ছেড়ে দেয়। না শেষ হলে তাকে Ready Queue-এর পেছনে পাঠানো হয়। এটি **Preemptive**।

বাস্তব উদাহরণ: তিনজন শিক্ষার্থীকে একবারে সর্বোচ্চ দুই মিনিট কথা বলার সুযোগ; যার কথা শেষ হয়নি সে আবার লাইনের শেষে দাঁড়ায়।

Time Quantum গুরুত্বপূর্ণ: খুব বড় হলে আচরণ FCFS-এর মতো হতে পারে; খুব ছোট হলে Context Switch বেশি হয়। Response Time ভালো হতে পারে, কিন্তু অতিরিক্ত Context Switch ও Waiting Time-এর অসুবিধা থাকতে পারে।

১০.১ Round Robin সম্পূর্ণ সমাধান — অতিরিক্ত তৈরি উদাহরণ

Time Quantum `q = 2 ms`।

Process	AT	BT	TQ
P1	0	5	2
P2	1	3	2
P3	2	1	2

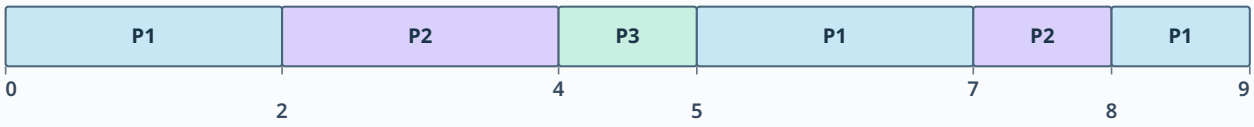
Tie Rule: কোনো Quantum শেষ হওয়ার ঠিক একই সময়ে নতুন Process এলে, আগে নতুন Arrival Ready Queue-এ ঢুকবে; এরপর অসমাপ্ত পুরোনো Process Queue-এর পেছনে যাবে।

সিদ্ধান্ত ১, 0–2: P1 প্রথমে। ২ সময় কাজ করল, বাকি `5−2=3`। এ সময় P2 সময় ১-এ এবং P3 সময় ২-এ এসেছে। নতুন Queue: `[P2, P3, P1]`।
সিদ্ধান্ত ২, 2–4: P2 সর্বোচ্চ ২ সময়; বাকি `3−2=1`। Queue: `[P3, P1, P2]`।
সিদ্ধান্ত ৩, 4–5: P3-এর প্রয়োজন ১ সময়, Quantum ২ হলেও ১ সময়েই শেষ। Queue: `[P1, P2]`।
সিদ্ধান্ত ৪, 5–7: P1 আবার ২ সময় পেল; তার বাকি `3−2=1`। Queue: `[P2, P1]`।

সিদ্ধান্ত ৫, 7–8: P2-এর বাকি ১ সময় শেষ। Queue: `[P1]`।

সিদ্ধান্ত ৬, 8–9: P1-এর শেষ ১ সময় শেষ।

Gantt Chart 11: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

Process	AT	BT	First Start	CT	TAT	WT	RT
P1	0	5	0	9	9−0 = **9**	9−5 = **4**	0−0 = **0**
P2	1	3	2	8	8−1 = **7**	7−3 = **4**	2−1 = **1**
P3	2	1	4	5	5−2 = **3**	3−1 = **2**	4−2 = **2**

Average TAT: `(9+7+3)/3 = 19/3 = 6.33`।

Average WT: `(4+4+2)/3 = 10/3 = 3.33`।

Average RT: `(0+1+2)/3 = 3/3 = 1.00`।

RT ও WT আলাদা বোঝা: P1 সঙ্গে সঙ্গে CPU পেয়েছে বলে RT=0, কিন্তু মাঝখানে অন্য Process-গুলো চলার সময় মোট ৪ সময় অপেক্ষা করেছে। তাই WT=4।

---

১১. পাঁচটি Algorithm এক টেবিলে

Algorithm	ধরন	CPU নির্বাচন	Preemption	সম্ভাব্য সমস্যা	সবচেয়ে ভালো দিক
FCFS	Non-Preemptive	আগে Arrival	না	Convoy Effect	সহজ ও সরল FIFO
SJF	এখানে Non-Preemptive	সবচেয়ে ছোট BT	না	বড় Job-এর Starvation	অনেক ক্ষেত্রে কম Average WT
SRTF	Preemptive	সবচেয়ে কম Remaining BT	হ্যাঁ	Starvation, Context Switch	ছোট Job দ্রুত শেষ হওয়ার সুযোগ
Priority	দুই ধরন	নির্ধারিত Priority	সংস্করণ অনুযায়ী	Low Priority Starvation	গুরুত্বপূর্ণ Process অগ্রাধিকার পায়
Round Robin	Preemptive	Ready Queue-এর পালা, q	Quantum শেষে	q খুব ছোট হলে Overhead	Interactive পরিবেশে Response উন্নত হতে পারে

১১.১ Convoy Effect বনাম Starvation

Convoy Effect: FCFS-এ বড় Process সামনে থেকে ছোট Process-গুলোকে অপেক্ষায় রাখে। উদাহরণ: BT ১৫-এর P1-এর পেছনে BT ২-এর P3।

Starvation: কোনো Process বারবার পিছিয়ে গিয়ে অত্যন্ত দীর্ঘ সময় CPU না পায়। SJF/SRTF-এ নতুন ছোট Job আসতে থাকলে বড় Job, কিংবা Priority-তে বেশি Priority-র Process আসতে থাকলে কম Priority-র Job এমন সমস্যায় পড়তে

পারে।

Aging: অপেক্ষার সময় বাড়ার সঙ্গে Priority বাড়িয়ে Starvation কমানোর কৌশল।

১১.২ দ্রুত Algorithm শনাক্ত করার নিয়ম

- **FCFS:** `AT` অনুসারে Ready Queue।
- **SJF:** `BT` সবচেয়ে ছোট, কেবল ইতিমধ্যে আসা Process; CPU নেওয়ার পর থামে না।
- **SRTF:** `Remaining BT` সবচেয়ে ছোট; নতুন Arrival-এ Preempt হতে পারে।
- **Priority:** `Priority` মান এবং উচ্চ/নিম্ন মানের দিকটি আগে দেখবে।
- **RR:** `q` দেখবে; প্রতিবার সর্বোচ্চ `q` সময়, তারপর Queue-এর পেছনে।

১২. একই Arrival হলে এবং CPU ফাঁকা থাকলে কী করবে?

সমান AT: FCFS-এ প্রশ্নে দেওয়া Tie Rule অনুসরণ করো। নিয়ম না থাকলে সাধারণত দেওয়া Process Order ব্যবহার করা হয়, তবে **নিয়মটি সমাধানের শুরুতে লিখবে**।

SJF-এ সমান BT: আগে AT, তারপর Process ID/প্রদত্ত ক্রম—যে Rule ধরে সমাধান করছ তা ঘোষণা করবে।

SRTF-এ সমান Remaining Time: অনেক ক্ষেত্রে Running Process অপরিবর্তিত রাখার নিয়ম ব্যবহার হয়; প্রশ্নের Rule প্রাধান্য পাবে।

Priority-তে সমান মান: Arrival Order/FCFS দিয়ে Tie ভাঙা যায়, যদি প্রশ্ন অন্য কিছু না বলে।

Round Robin-এ Boundary Arrival: Quantum শেষের ঠিক সময়ে Process আসলে আগে কে Queue-তে ঢুকবে, তা উল্লেখ করো।

CPU Idle: সবাই ভবিষ্যতে আসবে এবং এখন Ready Process নেই? পরবর্তী AT পর্যন্ত `Idle` দেখাবে। **Idle-এর দৈর্ঘ্য কোনো Process-এর BT নয়।**

১২.১ দুটি দ্রুত ধারণামূলক উদাহরণ

উদাহরণ A: P1 `AT=3, BT=4` এবং এর আগে আর কেউ নেই। Gantt: `Idle 0–3; P1 3–7`। এখানে `CT=7, TAT=7−3=4, WT=4−4=0`। CPU Idle থাকলেও P1 অপেক্ষা করেনি।

Gantt Chart 12: সময়ক্রম ও CPU ব্যবহার



চিত্রের প্রতিটি Block-এর দৈর্ঘ্য তার চলার সময়ের সমানুপাতিক; নিচের সংখ্যাগুলো সময়ের সীমা।

উদাহরণ B: P1 `AT=0, BT=6`; P2 `AT=2, BT=1`। **Non-Preemptive SJF**-এ `P1: 0–6, P2: 6–7` (P1 থামবে না)। **SRTF**-এ `P1: 0–2, P2: 2–3, P1: 3–7` (P1-এর Remaining=4, P2-এর BT=1, তাই P2 আগে)।

১৩. পরীক্ষায় অঙ্ক সমাধানের চেকলিস্ট

1. প্রশ্নের Algorithm এবং **Preemptive না Non-Preemptive** আলাদা করে লিখো।
2. `AT`, `BT`, `Priority`, `q`-এর ঘর তৈরি করো।

3. `t`-এ এখন পর্যন্ত আসা Process-গুলোই বিবেচনা করো।
4. প্রত্যেক Scheduling Decision-এর জন্য কারণ লেখো।
5. Gantt-এর Block-এর শেষ সময় = শুরু সময় + ওই Block-এ কাজের সময়।
6. Preemptive হলে একই Process একাধিক Block-এ আসতে পারে; **শেষ Block-এর শেষ সময়ই CT**।
7. প্রতিটি Process-এর `CT`, `TAT`, `WT`, `RT` গণনা করো।
8. Average করার আগে সব মান যোগ করে Process সংখ্যায় ভাগ করো।
9. CPU Idle এবং Tie Rule লুকিয়ে রেখো না।
10. শেষবার নিচের যাচাইগুলো করো।

১৩.১ শক্তিশালী গাণিতিক যাচাই

```
প্রতিটি Process-এর সব CPU Block-এর মোট দৈর্ঘ্য = তার BT
সব CPU Busy Block-এর সময়ের যোগফল = সব Process-এর BT-এর যোগফল
প্রথম থেকে শেষ সময় = CPU Busy Time + CPU Idle Time
প্রতিটি Process-এর TAT ≥ BT        (একক CPU Burst ও I/O-হীন অঙ্কে)
প্রতিটি Process-এর WT ≥ 0
প্রতিটি Process-এর RT ≥ 0
Non-Preemptive-এ RT = WT            (একক CPU Burst ও I/O-হীন অঙ্কে)
```

বিশেষ সতর্কতা: কোনো Process-এর `WT` ঋণাত্মক এলে সাধারণত Gantt বা সূত্র ভুল হয়েছে। `TAT`-কে BT ভেবে ফেলো না। RT বের করতে Process-এর **প্রথম** CPU পাওয়া সময় নেবে, শেষবারের সময় নয়।

১৪. সাধারণ ভুল ও তার সমাধান

ভুল	কেন ভুল	সঠিক নিয়ম
শুধু BT দেখে SJF শুরু করা	ছোট Process হয়তো এখনো আসেনি	বর্তমান সময়ে Ready থাকা Process দেখো
FCFS-এ কম BT আগে নেওয়া	FCFS BT নয়, Arrival দেখে	AT এবং Tie Rule অনুসরণ করো
SRTF-এ Original BT তুলনা	কিছু কাজ ইতিমধ্যে হয়ে গেছে	Remaining Time তুলনা করো
RR-এ সবসময় সম্পূর্ণ q দেওয়া	Process আগে শেষ হতে পারে	`min(q, Remaining BT)` চালাও
RR-এ Queue Order না লেখা	Boundary Arrival-এ ফল বদলাতে পারে	প্রতিটি Quantum শেষে Queue লেখো
Preemptive CT প্রথম Block থেকে নেওয়া	Process পুরো শেষ হয়নি	শেষ Block-এর শেষ সময় CT
Priority=1 সবসময় সবচেয়ে বড় ধরে নেওয়া	প্রশ্নভেদে উল্টো নিয়ম	Higher Priority কোন দিকে তা পড়ো
Idle-কে Waiting Time হিসেবে ধরে নেওয়া	Process তখনো আসেনি হতে পারে	WT কেবল Process-এর নিজের অপেক্ষা
Average-এ Process সংখ্যা ভুল	শেষ পর্যন্ত যোগফল ঠিক থাকলেও Average ভুল	মোট Process `n` দিয়ে ভাগ
Context Switch অতিরিক্ত যোগ করা	প্রশ্নে Overhead নাও থাকতে পারে	বলা না থাকলে সাধারণত বাদ দাও

অংশ C — বাংলাদেশে চাকরি পরীক্ষার উপযোগী অনুশীলন

১৫. Practice MCQ — এগুলো তৈরি করা অনুশীলনী, যাচাইকৃত বিগত প্রশ্ন নয়

MCQ ১. CPU Scheduling-এর প্রধান কাজ কী?
(A) Disk Format করা (B) Ready Process-এর মধ্যে CPU দেওয়ার জন্য নির্বাচন করা (C) File Rename করা (D) Password তৈরি করা
উত্তর: B। CPU Scheduler Ready Queue থেকে পরবর্তী Process নির্বাচন করে।

MCQ ২. FCFS সাধারণত কোন ধরনের Scheduling?
(A) Preemptive (B) Non-Preemptive (C) শুধু Deadline-Based (D) শুধু Priority-Based
উত্তর: B। FCFS-এ চলমান CPU Burst মাঝপথে Preempt করা হয় না।

MCQ ৩. SRTF-এ কোনটি তুলনা করা হয়?
(A) Process ID (B) Original Priority (C) Remaining CPU Time (D) Disk Size
উত্তর: C। ইতিমধ্যে CPU-তে কাজ করা সময় বাদ দিয়ে বাকি সময় দেখা হয়।

MCQ ৪. `AT=3, CT=12` হলে TAT কত?
(A) 3 (B) 9 (C) 12 (D) 15
উত্তর: B। `12−3=9`।

MCQ ৫. `TAT=10, BT=6` হলে WT কত?
(A) 4 (B) 6 (C) 10 (D) 16
উত্তর: A। `10−6=4`।

MCQ ৬. `AT=2, First Start=7` হলে RT কত?
(A) 2 (B) 5 (C) 7 (D) 9
উত্তর: B। `7−2=5`।

MCQ ৭. Round Robin-এর প্রয়োজনীয় Parameter কী?
(A) Time Quantum (B) Page Size (C) File Extension (D) Disk Track
উত্তর: A। Quantum একবারে CPU ব্যবহারের সর্বোচ্চ সময়।

MCQ ৮. FCFS-এর বড় কাজের পেছনে ছোট কাজ আটকে থাকার সমস্যার নাম কী?
(A) Aging (B) Convoy Effect (C) Thrashing (D) Deadlock Detection
উত্তর: B। দীর্ঘ Burst-এর কারণে ছোটগুলো অপেক্ষা করে।

MCQ ৯. Priority Scheduling-এ Starvation কমাতে কোন কৌশল ব্যবহার করা যায়?
(A) Aging (B) Formatting (C) Paging Only (D) Compiling
উত্তর: A। অপেক্ষা বাড়লে Priority উন্নত করা যায়।

MCQ ১০. Non-Preemptive SJF-এ `t=0`-এ একমাত্র P1 এসেছে এবং তার BT=9; P2 আসে `t=1`, BT=2। কে প্রথম চলবে?
(A) P2 (B) P1 (C) দুজন একসঙ্গে (D) কেউ নয়
উত্তর: B। P1 চালু হওয়ার সময় P2 আসেনি; P1 মাঝপথে থামবে না।

MCQ ১১. SRTF-এ চলমান Process-এর Remaining=7, নতুন Process-এর BT=3। সাধারণ নিয়মে কী হবে?
(A) একই চলবে (B) নতুনটি CPU পাবে (C) CPU Idle হবে (D) BT যোগ হবে
উত্তর: B। নতুনটির প্রয়োজনীয় CPU Time কম, তাই Preemption হয়।

MCQ ১২. RR-এ `q=4`, কোনো Process-এর Remaining=2। তার পরের CPU Block কত সময়?
(A) 4 (B) 6 (C) 2 (D) 0
উত্তর: C। কাজ শেষ করতে মাত্র ২ সময় লাগে।

MCQ ১৩. `CT=15, AT=4, BT=5` হলে WT কত?
(A) 6 (B) 11 (C) 5 (D) 15
উত্তর: A। `TAT=15−4=11`, `WT=11−5=6`।

MCQ ১৪. Preemptive অঙ্কে একই P1 তিনবার CPU পেলে CT কোথা থেকে নেওয়া হবে?
(A) প্রথম Block-এর শুরু (B) প্রথম Block-এর শেষ (C) শেষ Block-এর শেষ (D) Arrival Time
উত্তর: C। CT হলো সম্পূর্ণ কাজ শেষ হওয়ার সময়।

MCQ ১৫. `BT` যোগফল ১২, CPU Idle ৩, পর্যবেক্ষণ সময় `0–15` হলে CPU Utilization কত?
(A) 20% (B) 60% (C) 80% (D) 100%
উত্তর: C। `(12/15)×100=80%`।

MCQ ১৬. Non-Preemptive একক CPU Burst অঙ্কে `RT` সাধারণত কীসের সমান?
(A) BT (B) CT (C) WT (D) সব সময় 0
উত্তর: C। প্রথমবার CPU পাওয়ার আগে যতক্ষণ অপেক্ষা, সেটিই মোট অপেক্ষা।

MCQ ১৭. SJF-এ BT সমান হলে কী করতে হবে?
(A) সব সময় বড় PID আগে (B) প্রশ্নের Tie Rule মেনে চলতে হবে (C) CPU বন্ধ করতে হবে (D) কেবল Priority দেখতে হবে
উত্তর: B। Tie Rule স্পষ্ট না হলে অনুমানটি লিখতে হয়।

MCQ ১৮. `AT=5`, CPU `0–5` Idle, এরপর P1 `5–8` চলে; P1-এর WT কত?
(A) 5 (B) 8 (C) 3 (D) 0
উত্তর: D। P1 এসেই CPU পেয়েছে, আগের Idle তার অপেক্ষা নয়।

MCQ ১৯. কোন Algorithm-এ Time Quantum শেষ হলে অসমাপ্ত Process Ready Queue-এ ফিরে যায়?
(A) FCFS (B) Non-Preemptive SJF (C) RR (D) Non-Preemptive Priority
উত্তর: C। Round Robin পর্যায়ক্রমে CPU ভাগ করে দেয়।

MCQ ২০. তিনটি Process-এর WT যথাক্রমে ২, ৪ ও ৯ হলে Average WT কত?
(A) 3 (B) 5 (C) 9 (D) 15
উত্তর: B। `(2+4+9)/3=15/3=5`।

১৬. লিখিত পরীক্ষার জন্য গুরুত্বপূর্ণ প্রশ্ন ও সংক্ষিপ্ত উত্তর

প্রশ্ন ১: CPU Scheduling কী?
উত্তর: Operating System Ready Queue-এ থাকা একাধিক Process থেকে কোনটি পরবর্তী CPU পাবে, তা নির্বাচনের পদ্ধতিকে CPU Scheduling বলে। লক্ষ্য হলো CPU-এর কার্যকর ব্যবহার এবং প্রয়োজন অনুযায়ী Waiting, Turnaround ও Response Time উন্নত করা।

প্রশ্ন ২: Non-Preemptive ও Preemptive-এর পার্থক্য লেখো।
উত্তর: Non-Preemptive পদ্ধতিতে চলমান CPU Burst মাঝপথে কেড়ে নেওয়া হয় না; Preemptive পদ্ধতিতে Scheduling নিয়ম অনুযায়ী চলমান Process সাময়িক থামানো যায়। FCFS এবং Non-Preemptive SJF প্রথম ধরনের; SRTF ও RR দ্বিতীয় ধরনের।

প্রশ্ন ৩: FCFS এবং SJF-এর দুটি প্রধান পার্থক্য লেখো।
উত্তর: FCFS Arrival Order দেখে, SJF Ready Process-এর মধ্যে কম Burst Time দেখে। FCFS-এ Convoy Effect হতে পারে; SJF-এ বড় Job-এর Starvation হতে পারে।

প্রশ্ন ৪: FCFS-এ Convoy Effect কী?
উত্তর: বড় CPU Burst-ওয়ালা Process আগে CPU পেলে তার পেছনে থাকা ছোট Process-গুলো দীর্ঘক্ষণ অপেক্ষা করে। এই সারিবদ্ধ অপেক্ষার প্রভাবকে Convoy Effect বলে।

প্রশ্ন ৫: SRTF এবং SJF কীভাবে আলাদা?
উত্তর: Non-Preemptive SJF-এ Process CPU পেলে Burst শেষ না হওয়া পর্যন্ত চলে; SRTF-এ কম Remaining Time-এর Process এসে চলমানটিকে Preempt করতে পারে।

প্রশ্ন ৬: Time Quantum ছোট হলে কী হয়?
উত্তর: Process-গুলোর মধ্যে CPU ঘন ঘন বদলায়। এতে প্রথম Response দ্রুত হওয়ার সুযোগ থাকলেও Context Switch Overhead বেড়ে সামগ্রিক দক্ষতা কমতে পারে।

প্রশ্ন ৭: Starvation ও Aging বুঝিয়ে লেখো।
উত্তর: কোনো Process অনেক দীর্ঘ সময় CPU না পাওয়াকে Starvation বলে। অপেক্ষার সময় অনুযায়ী Priority বাড়িয়ে তাকে CPU পাওয়ার সুযোগ দেওয়াকে Aging বলে।

প্রশ্ন ৮: CT, TAT, WT ও RT-এর সূত্র লেখো।
উত্তর: `TAT=CT−AT; WT=TAT−BT; RT=First Start−AT`। CT সরাসরি Gantt Chart-এ Process-এর কাজ শেষ হওয়ার সময় থেকে নেওয়া হয়।

১৬.১ নিজে সমাধান করো — অনুশীলনী (শিক্ষকের নোট নয়)

অনুশীলন A (FCFS): P1(AT=0,BT=4), P2(AT=1,BT=2), P3(AT=3,BT=1)। Gantt, CT, Average WT বের করো।
যাচাই উত্তর: `P1 0–4; P2 4–6; P3 6–7`; CT = `(4,6,7)`; WT = `(0,3,3)`; Average WT= `2.00`।

অনুশীলন B (Non-Preemptive SJF): P1(AT=0,BT=6), P2(AT=0,BT=2), P3(AT=1,BT=3); একই AT-এ তালিকার ক্রম মেনে চলো।
যাচাই উত্তর: `P2 0–2; P3 2–5; P1 5–11`; WT = `(5,0,1)`; Average WT= `2.00`।

অনুশীলন C (SRTF): P1(AT=0,BT=5), P2(AT=1,BT=2), P3(AT=3,BT=1)। সমান Remaining Time-এ Running Process চালাও।
যাচাই উত্তর: `P1 0–1; P2 1–3; P3 3–4; P1 4–8`; CT=(8,3,4); WT=(3,0,0); Average WT= `1.00`।

অনুশীলন D (Round Robin): P1(AT=0,BT=3), P2(AT=0,BT=2), `q=1`, Queue শুরুতে P1 → P2।
যাচাই উত্তর: `P1 0–1; P2 1–2; P1 2–3; P2 3–4; P1 4–5`; CT=(5,4); WT=(2,2); Average WT= `2.00`।

১৭. বিশেষ Revision Sheet — পরীক্ষা শুরুর আগে একবার দেখো

মনে রাখার বিষয়	এক লাইনের সূত্র / কৌশল
AT	Process যখন Ready Queue-তে আসে
BT	মোট CPU-তে কাজ করার সময়
CT	শেষবার CPU থেকে কাজ শেষ করে বের হওয়ার সময়
TAT	`CT − AT`
WT	`TAT − BT` (এক CPU Burst, I/O-হীন অঙ্কে)
RT	`First Start − AT`
Average	সব মানের যোগফল ÷ Process সংখ্যা
FCFS	আগে Arrival → আগে CPU
SJF	উপস্থিত Process-এর কম BT
SRTF	উপস্থিত Process-এর কম Remaining BT; Preemption হতে পারে
Priority	প্রশ্নে নির্ধারিত উচ্চ Priority আগে
RR	একবারে সর্বোচ্চ `q`, অসমাপ্ত হলে Queue-এর পেছনে
CPU Idle	Ready Process নেই, তাই সময়রেখায় `Idle`
Tie	শুরুতেই Tie Rule ঘোষণা করো
Preemptive CT	শেষ CPU Block-এর **শেষ সময়**
মূল যাচাই	মোট CPU Busy Time = মোট BT

১৮. শিক্ষকের নোটের অঙ্কগুলোর উত্তর এক নজরে

উৎস ও অঙ্ক	Gantt-এ Execution Order	Average TAT	Average WT
পৃষ্ঠা ৩, FCFS-১	P1 → P2 → P3	**18.33**	**11.33**
পৃষ্ঠা ৪, FCFS-২	P3 → Idle → P1 → P4 → P2 → P5	**7.80**	**3.80**
পৃষ্ঠা ৪, FCFS-৩	P4 → P3 → P1 → P5 → P2	**12.60**	**8.00**
পৃষ্ঠা ৫, SJF-১	P3 → P2 → P1	**9.67**	**2.67**
পৃষ্ঠা ৫, SJF-২	P2 → P5 → P3 → P1 → P4	**8.20**	**3.80**

এখানে শেষবার মনে রাখবে: অঙ্কের প্রথম কাজ Formula লেখা নয়; **প্রথমে ঠিক Process নির্বাচন করে সঠিক Gantt Chart আঁকা**। Gantt ঠিক হলে CT পাওয়া সহজ, আর CT ঠিক হলে TAT, WT, RT প্রায় যান্ত্রিকভাবে বের হয়ে যাবে।

উৎস-সীমা ও সম্পাদনা নোট: শিক্ষকের পৃষ্ঠা ১–৫ থেকে বিষয়, সংখ্যাগুলো ও FCFS/SJF Gantt ক্রম নেওয়া হয়েছে। এই হ্যান্ডনোটে দেখানো বিশদ CT/TAT/WT/RT টেবিল, গাণিতিক যাচাই, SRTF/Priority/RR অঙ্ক, MCQ, উপমা ও পরীক্ষার কৌশল শেখার সুবিধার্থে নতুনভাবে যোগ করা। শিক্ষকের নোটের অনুপস্থিত তথ্যকে শিক্ষক-প্রদত্ত হিসেবে দেখানো হয়নি। সব অঙ্কে I/O Burst এবং Context Switch Overhead দেওয়া নেই বলে আলাদা সময় যোগ করা হয়নি।